EngiRent AI Verification System — Complete Technical Documentation

Project: EngiRent Hub — Smart Kiosk Equipment Rental System **Institution:** University of Cebu Lapu-Lapu and Mandaue (UCLM), College of Engineering **Service Location:** `server/python_server/services/ml/` **Language:** Python 3.12 **Framework:** FastAPI **Port:** 8001

Table of Contents

1. Overview
2. Two Verification Checkpoints
3. System Architecture
4. Image Pre-processing
5. The 8-Stage Verification Pipeline
    - Stage 1: Image Quality Gate
    - Stage 2: Perceptual Hash Pre-filter
    - Stage 3: Traditional Computer Vision
    - Stage 4: SIFT + FLANN + RANSAC
    - Stage 5: SSIM Structural Similarity
    - Stage 6: ResNet50 Deep Learning
    - Stage 7: OCR Serial Number Matching
    - Stage 8: Hybrid Final Score
6. Decision Flow
7. Libraries — Complete Reference
8. What Is Unique to This Project
9. Configuration Reference
10. API Endpoints
11. API Response Structure
12. Feature Caching
13. Backend Integration
14. Deployment
15. File Structure

---

1. Overview

The AI Verification Service is a standalone **Python microservice** whose sole responsibility is to answer one question:

> ***Is the physical item placed inside the kiosk locker the same item shown in the reference photos?***

It is called by the Node.js backend (`server/src/controllers/kioskController.ts`) at two critical moments in the rental lifecycle. It runs as a separate process on port 8001, completely decoupled from the main API.

What It Is Not

- It is **not** a general object classifier (it does not say "this is a laptop")
- It is **not** a face recognition system (that is separate hardware-level logic)
- It does **not** store data — it receives images, computes a similarity score, and returns a decision
- It is **not** a YOLOv8 detector, despite early documentation mentioning it

What It Does

Given two sets of images — **reference images** (uploaded by the owner when listing the item) and **kiosk images** (captured by the camera inside the locker) — it runs 8 different similarity algorithms, combines their scores with a weighted formula, and returns one of four decisions: `APPROVED`, `PENDING`, `RETRY`, or `REJECTED`.

---

2. Two Verification Checkpoints

The service is called at two different points during a rental transaction, protecting both parties against fraud.

Checkpoint 1 — Owner Deposits the Item

Trigger: Owner arrives at the kiosk and places the item in a locker before the renter picks it up.

Question being answered: Is this the same item the owner photographed and listed? Or has the owner swapped it for a different/inferior item?

Who is protected: The **renter** — they paid based on the listing photos. They should receive exactly that item.

On failure (REJECTED): The rental is cancelled and the renter is refunded. The owner cannot proceed.

Checkpoint 2 — Renter Returns the Item

Trigger: Renter returns to the kiosk at the end of the rental period and places the item back in a locker.

Question being answered: Is this the same item that was rented out? Or has the renter swapped it for a damaged, broken, or different item?

Who is protected: The **owner** — their item should be returned in the same condition.

On failure (REJECTED): The rental moves to `DISPUTED` status (not cancelled, since the renter still has the item). An admin reviews the case.

Summary

Checkpoint	Who deposits	Reference images	Kiosk images	Protects
Deposit	Owner	Item listing photos	Locker camera at deposit	Renter
Return	Renter	Item listing photos	Locker camera at return	Owner

---

3. System Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                    Node.js Backend (port 5000)                 │
│                                                                │
│  kioskController.ts                                            │
│  ├── depositItem()  ─── POST /api/v1/verify ──────────────┐   │
│  └── returnItem()   ─── POST /api/v1/verify ──────────┐   │   │
│                                                       │   │   │
│  itemController.ts                                    │   │   │
│  └── createItem()   ─── POST /api/v1/extract-features ─│───│───│
└──────────────────────────────────────────────────────────│───│───┘
```

```
                                        │   │
                              ┌─────────────────────────────▼───▼───┐
                              │   Python ML Service (port 8001)      │
                              │                                      │
                              │   FastAPI app (main.py)              │
                              │   └── /api/v1/verify                 │
                              │   └── /api/v1/extract-features       │
                              │   └── /api/v1/health                 │
                              │                                      │
                              │   HybridVerifier (hybrid.py)         │
                              │   ├── Stage 1: Quality gate          │
                              │   ├── Stage 2: pHash pre-filter      │
                              │   ├── Stage 3: Traditional CV        │
                              │   ├── Stage 4: SIFT + RANSAC         │
                              │   ├── Stage 5: SSIM                  │
                              │   ├── Stage 6: ResNet50              │
                              │   ├── Stage 7: OCR                   │
                              │   └── Stage 8: Hybrid score          │
                              └──────────────────────────────────────┘
                                              │
                              ┌───────────────▼──────────────┐
                              │   MySQL Database             │
                              │   Verification table         │
                              │   Item.mlFeatures (cache)    │
                              └──────────────────────────────┘
```

---

4. Image Pre-processing

Before any algorithm runs, every image goes through a normalisation pipeline (`utils/image.py` , `utils/background.py`) to reduce differences between the owner's home photo environment and the kiosk locker environment.

4a. Loading (`utils/image.py` — `load_image()`)

Accepts three input formats:

- File path string → `cv2.imread()`
- Raw bytes → `cv2.imdecode(np.frombuffer(...))`
- NumPy array (BGR) → used as-is

4b. White Balance (`utils/image.py` — `white_balance()`)

Algorithm: Gray-world assumption.

The assumption: on average, the colors in any natural scene should be neutral gray. If the mean of the red channel differs from the mean of the blue channel, the lighting is tinted and should be corrected.

```
scale_r = overall_mean / mean_red
scale_g = overall_mean / mean_green
scale_b = overall_mean / mean_blue
```

Each channel is multiplied by its scale, then clipped to [0, 255].

Why: Owner photographs are taken under varied indoor lighting (warm/cool white bulbs). The kiosk locker uses its own fixed lighting. Without correction, the same item can appear significantly different in color between the two environments.

4c. Lighting Normalisation (`utils/image.py` — `normalize_lighting()`)

Algorithm: CLAHE — Contrast Limited Adaptive Histogram Equalization.

Steps:

1. Convert BGR → LAB color space (separates luminance from color)
2. Apply CLAHE only to the L (luminance) channel
3. Convert back to BGR

CLAHE divides the image into small tiles and equalizes the histogram within each tile, with a contrast limit to prevent over-amplification of noise.

Why: Standard global histogram equalisation can over-brighten shadows or blow out highlights. CLAHE applies it locally (tile-by-tile) with contrast limits, resulting in more natural-looking normalisation.

4d. Resizing (`utils/image.py` — `resize_image()`)

Target size: **640 × 640** (configurable via `TARGET_SIZE`).

Maintains aspect ratio: the image is scaled so the longest dimension fits, then zero-padded on the shorter dimension to reach 640×640. This avoids distortion.

4e. Background Removal (`utils/background.py`)

This is the most impactful pre-processing step for this project.

Problem: The owner photographs the item on a wooden desk, carpet, or white surface at home. The kiosk camera photographs it against a bright white locker interior. If the background is included in feature extraction, the comparison would partially measure "desk vs. locker wall" instead of "item vs. item."

Two methods:

Method 1 — GrabCut (for general/listing images): `remove_background_grabcut()`

GrabCut is an iterative segmentation algorithm that alternates between:

1. Estimating a Gaussian Mixture Model (GMM) for foreground and background pixels
2. Assigning each pixel to foreground or background based on the GMMs and graph cuts

Implementation details:

- Initialised with a center-biased rectangle covering 70% of the image
- 5 iterations of the GrabCut algorithm
- Post-processing: `cv2.MORPH_CLOSE` (fills small holes in foreground) then `cv2.MORPH_OPEN` (removes small noise islands)
- Result: a binary mask where 1 = item, 0 = background

Method 2 — White threshold (for kiosk images): `remove_background_kiosk()`

Since the kiosk locker interior is white (V > 200 in HSV), a simpler approach:

1. Convert to HSV
2. All pixels with V > 200 are classified as background

3. Optional: if an empty-locker reference image is available, use absolute difference subtraction first

After background removal, `get_item_crop()` extracts the tight bounding box around the detected foreground, with 10px padding.

---

5. The 8-Stage Verification Pipeline

Source file: `app/comparison/hybrid.py` — `HybridVerifier.verify()`

The pipeline processes all combinations of original × kiosk image pairs. With 3 original and 3 kiosk images, this is 9 pairwise comparisons per method.

Stage 1: Image Quality Gate

Source file: `app/utils/quality.py` — `check_quality()`

Purpose: Reject blurry, dark, or item-absent images before running expensive algorithms. A bad kiosk image would produce meaningless scores, wasting compute and potentially causing false rejections.

Three checks:

Check 1 — Blur Detection (Laplacian Variance)

```
blur_score = cv2.Laplacian(gray, cv2.CV_64F).var()
```

The Laplacian operator computes the second derivative of pixel intensity. Sharp images have strong edges with high second-derivative values. Blurry images have smoothed edges and low variance.

- **Threshold:** `QUALITY_MIN_BLUR_SCORE = 50.0`
- **Fails if:** `blur_score < 50.0`

Check 2 — Brightness

```
brightness = gray.mean()
```

Simple mean pixel value of the grayscale image.

- **Range:** `QUALITY_MIN_BRIGHTNESS = 40.0` to `QUALITY_MAX_BRIGHTNESS = 240.0`
- **Fails if:** `brightness < 40` (too dark, item not visible) or `brightness > 240` (overexposed, all white)

Check 3 — Item Coverage

```
coverage = foreground_pixels / total_pixels
```

Uses the foreground mask from background removal. Ensures the item actually takes up a meaningful portion of the frame.

- **Threshold:** `QUALITY_MIN_COVERAGE = 0.05` (item must cover at least 5% of the frame)
- **Fails if:** `coverage < 0.05`

Pipeline action: If **all** kiosk images fail the quality gate, return `RETRY` immediately without running any further stages. Partial failures (some images pass, some fail) are allowed — the passing images continue through the pipeline.

Stage 2: Perceptual Hash Pre-filter

Source file: `app/features/phash.py`

Purpose: Fast, O(1) pre-screening. If two items are completely different (a calculator vs. a lab gown), this stage catches it in milliseconds and skips the expensive 5-second pipeline entirely.

Two hash methods are computed:

pHash — Perceptual Hash (DCT-based)

1. Convert to grayscale
2. Resize to 32×32
3. Apply **DCT (Discrete Cosine Transform)** to the entire 32×32 image — same transform used in JPEG compression
4. Keep only the top-left **16×16 block** (low-frequency components representing dominant visual patterns)
5. Compute median of the 256 DCT values
6. **Binary hash:** each of 256 values becomes 1 if above median, 0 if below
7. Result: a 256-bit fingerprint of the image's dominant visual structure

dHash — Difference Hash (gradient-based)

1. Convert to grayscale
2. Resize to 17×16
3. Compare each pixel to the pixel immediately to its right: 1 if `pixel[x] > pixel[x+1]`, else 0
4. Result: a 256-bit fingerprint of horizontal gradient patterns

Comparison

```
similarity = (1 - hamming_distance(hash1, hash2) / total_bits) × 100
```

Hamming distance counts the number of bit positions where the two hashes differ. Identical images → 0 distance → 100% similarity. Completely different images → ~128/256 bits differ (random) → ~50% similarity.

Pipeline action: If **all** original×kiosk image pairs score below `PHASH_OBVIOUS_MISMATCH_THRESHOLD = 40.0%`, return `RETRY` / `REJECTED` immediately. This threshold is intentionally low — it only catches obvious mismatches. If one pair has any visual overlap at all, the pipeline continues.

Best pHash score is recorded and used in the final hybrid score (10% weight).

Stage 3: Traditional Computer Vision

Source files: `app/features/traditional.py`, `app/comparison/similarity.py`

Purpose: Extract explicit, interpretable visual features from the item and compare them numerically. This stage computes 6 types of features, each capturing a different visual property.

Background removal runs before this stage. All features are extracted from the foreground-only item crop.

Feature 3a — HSV Color Histogram (84 dimensions)

Algorithm: Color distribution in HSV (Hue-Saturation-Value) color space.

```
H channel: 36 bins (0-179 in OpenCV → 5° resolution per bin)
S channel: 32 bins (0-255)
```

```
V channel: 16 bins (0-255) — fewer bins, brightness matters less
Total: 84-dimensional vector
```

Normalised so all bins sum to 1 (probability distribution).

Why HSV instead of RGB? RGB mixes color and brightness together — the same red object looks darker or lighter depending on lighting. HSV separates them: H (hue) is the actual color, V (value) is brightness. By using H with many bins and V with few bins, the feature is robust to lighting changes.

Comparison method: Cosine similarity — measures the angle between the two 84-d vectors regardless of their magnitudes.

Feature 3b — Spatial Color Pyramid (108 dimensions)

Algorithm: Divide the image into a 3×3 grid, compute a compact color histogram in each of the 9 cells.

```
3 rows × 3 columns = 9 cells
Each cell: 12-bin hue histogram
Total: 9 × 12 = 108 dimensions
```

Why spatial? A global color histogram cannot distinguish an item where blue is on top and black on the bottom from one where black is on top and blue on the bottom — both have the same global histogram. The spatial pyramid captures **where** colors are located in the image.

Comparison method: Cosine similarity on the full 108-d vector.

Feature 3c — Shape via Hu Moments (7 dimensions)

Algorithm:

1. Canny edge detection (thresholds: 50, 150) on grayscale image
2. Find all external contours
3. Select the largest contour by area (the item's outline)
4. Compute raw image moments `M_pq = Σ Σ x^p · y^q · I(x,y)`
5. Derive central and normalized moments
6. Compute 7 Hu Moments (mathematical invariants)
7. Log-transform: `-sign(h) × log10(|h| + 1e-10)` to normalize scale

What are Hu Moments? Seven values that describe the shape's geometry — roughly: elongation, symmetry, skewness, and higher-order shape properties. They are **rotation-invariant** (the same shape rotated 90° gives the same Hu moments) and **scale-invariant**.

Why useful? An Arduino board is rectangular. A lab gown is roughly trapezoidal. A power bank is rectangular but very elongated. These shapes have distinct Hu moment signatures.

Comparison method: Inverse distance: `1 / (1 + |a - b|)` — maps distance 0 to similarity 1.0, distance ∞ to similarity 0.

Feature 3d — Multi-Scale LBP Texture (62 dimensions)

Algorithm: Local Binary Pattern at three scales.

For each pixel, compare it to `P` neighbours arranged in a circle of radius `R`:

- If neighbour is ≥ centre pixel: bit = 1

- If neighbour is < centre pixel: bit = 0
- Result: a P-bit binary number (the LBP code)

Three scales:

```
R=1, P=8  → 10 uniform pattern bins  (micro-texture: pixel-level patterns)
R=2, P=16 → 18 uniform pattern bins  (medium-texture: grain, weave)
R=4, P=32 → 34 uniform pattern bins  (macro-texture: fabric structure)
Total: 62 dimensions
```

Uniform patterns (at most 2 bit transitions, e.g. `00001111`) represent meaningful textures like edges, flat regions, and corners. Non-uniform patterns are grouped into a single bin.

Why multi-scale? Single-scale LBP only captures micro-texture. Multi-scale captures the texture at three different levels of zoom — important for distinguishing fabric weave from solid plastic from rough metal.

What does texture detect? Smooth electronics vs. textured lab gown fabric vs. rough circuit board.

Comparison method: Pearson correlation — measures the linear relationship between the two 62-d texture distributions.

Feature 3e — HOG Edge Gradients (variable dimensions)

Algorithm: Histogram of Oriented Gradients.

1. Resize image to 128×128
2. Compute image gradients (magnitude and direction) at every pixel
3. Divide into 8×8 cells, then group cells into 16×16 blocks
4. In each cell: build a 9-bin histogram of gradient directions (0°–180°, 20° bins)
5. Normalise across overlapping 2×2 block windows

HOGDescriptor configuration:

```
window:      128 × 128
block:       32 × 32 (4×4 cells)
block stride: 16 × 16
cell:        16 × 16
bins:        9
```

Why HOG? HOG captures structural details: the outline of a screen bezel, the position of buttons on a calculator, the shape of a USB port, the collar line of a lab gown. Unlike pixel-level features, HOG is robust to lighting changes because it uses gradient magnitudes rather than absolute pixel values.

Comparison method: Cosine similarity. The output vector is L2-normalised before comparison.

Feature 3f — ORB Keypoint Descriptors (N × 32 binary matrix)

Algorithm: ORB (Oriented FAST and Rotated BRIEF)

1. **FAST** corner detector finds up to 200 keypoints (strong corners, edges)
2. **Harris** score ranks and selects the best 200
3. **Orientation** assigned using intensity centroid in a circular patch
4. **BRIEF** descriptor: 256 bit-comparisons between pre-defined pixel pairs in a 31×31 patch, rotated to match keypoint orientation
5. Result: each keypoint has a 32-byte (256-bit) binary descriptor

Comparison: BFMatcher with **Hamming distance** (fast for binary descriptors) + **Lowe's ratio test**:

```
For each match m with nearest neighbour n:
  Accept if: m.distance < 0.75 × n.distance
```

The ratio test filters ambiguous matches — if the nearest neighbour is not clearly better than the second-nearest, the match is discarded.

```
orb_similarity = good_matches / min(descriptors_A, descriptors_B)
```

Why ORB instead of SIFT here? ORB is much faster for real-time use and is patent-free. It is used in this traditional stage for quick descriptor matching. The more robust (but slower) SIFT with RANSAC has its own dedicated Stage 4.

Traditional Stage Final Score

All 6 sub-scores combined:

```
traditional_score = (
    color_sim    × 0.22 +
    spatial_sim  × 0.13 +
    shape_sim    × 0.12 +
    texture_sim  × 0.13 +
    hog_sim      × 0.18 +
    orb_sim      × 0.12
)
```

This is computed for every original×kiosk image pair. The set of scores is then aggregated (trimmed mean — see Stage 8).

Stage 4: SIFT + FLANN + RANSAC

Source file: `app/features/sift.py`

Purpose: Find geometric keypoint correspondences between the two images and verify that they are physically consistent. This is the most mathematically rigorous stage — RANSAC geometrically validates that matched keypoints are not random coincidences.

Step 1 — SIFT Keypoint Detection

SIFT (Scale-Invariant Feature Transform) detects and describes local image features.

How keypoints are found:

1. Build a **Gaussian scale-space**: progressively blur the image at different scales
2. Compute **Difference of Gaussians (DoG)**: subtract adjacent blurred images
3. Find local extrema in the DoG scale-space (points that are maximal/minimal both spatially and across scales)
4. Refine location to sub-pixel accuracy
5. Assign orientation from local gradient histogram
6. Compute a 128-dimensional descriptor: 4×4 grid of 8-bin gradient histograms around the keypoint

Scale and rotation invariance: Because the descriptor is built from the DoG scale-space and aligned to the dominant gradient orientation, the same point on an item produces the same 128-d vector regardless of how large or tilted the item is in the photo.

Step 2 — FLANN Matching

FLANN (Fast Library for Approximate Nearest Neighbors) with KD-tree index:

```
index_params = dict(algorithm=1, trees=5)   # FLANN_INDEX_KDTREE
search_params = dict(checks=50)
```

For each descriptor in image A, find the 2 nearest descriptors in image B (k=2 nearest neighbour search).

Step 3 — Lowe's Ratio Test

For each candidate match (nearest `m`, second-nearest `n`):

```
Accept if: m.distance < 0.7 × n.distance
```

This filters ambiguous matches. If the nearest match is not clearly better than the second-nearest, the point is non-distinctive and the match is discarded. Lowe's original 2004 paper established 0.7–0.8 as optimal.

Step 4 — RANSAC Homography

With ≥4 good matches, RANSAC (Random Sample Consensus) verifies geometric consistency:

1. Randomly sample 4 point correspondences
2. Compute the **homography matrix** H (a 3×3 matrix representing a perspective transformation)
3. Check how many of the remaining matches are **inliers** (their projected positions match within 5 pixels)
4. Repeat many times, keep the H with the most inliers

```
_, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
inlier_count = mask.sum()
inlier_ratio = inlier_count / len(good_matches)
```

Why RANSAC? Without it, random false matches could inflate the score. RANSAC proves that the matched keypoints are geometrically consistent — they all fit a single perspective transformation. Random matches cannot fit a consistent transformation.

SIFT Final Score

```
sift_score = inlier_ratio × 0.7 + match_ratio × 0.3
```

Inlier ratio (RANSAC-verified) is weighted more heavily because it is geometrically validated. Raw match ratio is included as a secondary signal for cases with few keypoints where RANSAC cannot run.

Stage 5: SSIM Structural Similarity

Source file: `app/comparison/similarity.py` — `compare_ssim()`, `_compute_ssim()`

Algorithm: SSIM — Structural Similarity Index Measure (Wang et al., 2004).

SSIM models the human visual system's perception of image quality. Rather than measuring pixel-level differences, it compares three perceptual components:

1. **Luminance** — are the average brightness levels similar?
2. **Contrast** — are the local contrast patterns similar?

3. **Structure** — do the local spatial patterns correlate?

Implementation:

Images are resized to 256×256 and converted to grayscale. Then a sliding 11×11 Gaussian window (σ=1.5) moves across both images:

```
mu1 = GaussianBlur(img1, (11,11), 1.5)       # local mean
mu2 = GaussianBlur(img2, (11,11), 1.5)
sigma1_sq = GaussianBlur(img1², (11,11), 1.5) - mu1²    # local variance
sigma2_sq = GaussianBlur(img2², (11,11), 1.5) - mu2²
sigma12   = GaussianBlur(img1*img2, (11,11), 1.5) - mu1*mu2  # covariance

ssim_map = ((2*mu1*mu2 + C1) * (2*sigma12 + C2)) /
           ((mu1² + mu2² + C1) * (sigma1_sq + sigma2_sq + C2))
```

Where `C1 = (0.01×255)²` and `C2 = (0.03×255)²` are stability constants to avoid division by zero.

The final SSIM score is the mean over all pixel windows. Range: [-1, 1], clamped to [0, 100%].

Why SSIM? Traditional pixel-level metrics (MSE, PSNR) are sensitive to small spatial misalignments. SSIM is more robust — two images that are shifted by a few pixels but show the same object will have a high SSIM score.

Stage 6: ResNet50 Deep Learning

Source file: `app/features/deep.py`

Algorithm: Feature extraction using a pre-trained ResNet50 convolutional neural network.

The Model

ResNet50 (Residual Network with 50 layers) was trained on **ImageNet** — a dataset of 1.2 million images across 1,000 categories.

The full ResNet50 architecture ends with a fully-connected classification layer that outputs 1,000 class probabilities. For feature extraction, that final layer is **removed**:

```
resnet = models.resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)
model = torch.nn.Sequential(*list(resnet.children())[:-1])  # Remove last layer
```

The penultimate layer is an **average pooling** layer that collapses the spatial dimensions of the final convolutional feature maps into a **2048-dimensional vector**. This vector is a compact numerical representation of "what this image looks like" at a high semantic level.

Why ResNet50?

ResNet (He et al., 2015) introduced **residual connections** (skip connections): instead of learning `H(x)`, each block learns `F(x) = H(x) - x`, making the output `H(x) = F(x) + x`. This allows gradients to flow directly through the network during training, enabling much deeper networks (50+ layers) without vanishing gradient problems.

The V2 weights (`IMAGENET1K_V2`) are the second-generation ImageNet weights with improved training procedures.

Input Pre-processing

```
transforms.Resize(256)
transforms.CenterCrop(224)
transforms.ToTensor()
transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
```

These are the standard ImageNet normalisation parameters. All PyTorch/TorchVision models trained on ImageNet expect this exact normalisation.

Lazy Loading

The model is loaded only on first call (`_load_model()` caches `_model` and `_transform` as globals). This avoids a ~2-second startup delay when the service starts. It also means the model is only loaded if `ENABLE_DEEP_LEARNING=true` .

Inference

```
with torch.no_grad():
    features = model(tensor)  # shape: (1, 2048, 1, 1)
return features.squeeze().numpy()  # shape: (2048,)
```

`torch.no_grad()` disables gradient computation — not needed for inference, saves memory and compute.

Comparison

Cosine similarity between two 2048-d vectors:

```
similarity = 1 - cosine_distance(vector_A, vector_B)
```

Two images of the same object (even from different angles or lighting) will have similar 2048-d vectors because ResNet50 learned to recognise the same objects under different conditions from ImageNet.

What Deep Learning Captures That Traditional CV Cannot

- **Semantic similarity**: recognises that two photos show the same type of object
- **Part correspondence**: matches regions with similar visual semantics (screen area, keyboard region, connector ports)
- **Appearance invariance**: trained to handle varied lighting, backgrounds, and viewpoints
- **Brand/model recognition**: subtly encodes brand logos and product design characteristics

---

Stage 7: OCR Serial Number Matching

Source file: `app/utils/ocr.py`

Purpose: If an item has a visible serial number, model number, or branded text, match it as a hard verification signal. Two identical-looking laptops with different serial numbers are definitely not the same device.

Pre-processing Before OCR

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
blur = cv2.GaussianBlur(gray, (3,3), 0)
```

```
thresh = cv2.adaptiveThreshold(blur, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                               cv2.THRESH_BINARY, 11, 2)
```

Adaptive thresholding handles varying illumination across the image — text on a label with slightly uneven lighting is still binarised cleanly.

OCR Engine

pytesseract is a Python wrapper around **Google's Tesseract OCR engine** — the most widely used open-source OCR engine, developed originally by HP and now maintained by Google.

```
text = pytesseract.image_to_string(thresh)
```

Pattern Matching

Four regex patterns extract structured identifiers:

```
r'[Ss]/[Nn][:\s]*([A-Z0-9\-]{4,})'     # S/N: ABC123
r'[Ss]erial[:\s]*([A-Z0-9\-]{4,})'     # Serial: XYZ-456
r'[Mm]odel[:\s]*([A-Z0-9\-]{4,})'      # Model: MDL-001
r'\b([A-Z0-9]{6,})\b'                   # Generic 6+ char alphanumeric
```

Extracted text is uppercased and deduplicated. Strings shorter than 4 characters are filtered out.

Matching

`match_serial_numbers()` compares the set of serials from original images against the set from kiosk images. If any serial number appears in both sets → `ocr_match = True`.

Effect on final score:

```
if ocr_match:
    final_score = min(100.0, final_score + 10.0)  # +10% bonus
```

This is additive — it cannot single-handedly approve a verification, but it provides meaningful positive evidence when a serial number is visible and matches.

Stage 8: Hybrid Final Score

Source file: `app/comparison/hybrid.py` — `verify()` lines 226–292

This stage combines all method scores into a single confidence value and maps it to a decision.

Score Aggregation

For each method (traditional, SSIM, deep), there are multiple pairwise scores (one per original×kiosk image combination). These are aggregated before combining methods.

Aggregation method: Trimmed mean (default):

```
sorted_scores = sorted(scores)
n = len(sorted_scores)
low_cut  = max(1, int(n * 0.20))            # drop bottom 20%
```

```
high_cut = max(low_cut+1, n - max(1, int(n * 0.10)))  # drop top 10%
trimmed  = sorted_scores[low_cut:high_cut]
result   = mean(trimmed)
```

Why not plain max? A single good image pair could game the system — the owner takes one very clean photo and the rest are poor. Trimmed mean prevents any single outlier (high or low) from dominating.

Why not median? Median is robust but loses information. Trimmed mean preserves more signal while still being outlier-resistant.

Hybrid Score Formula

When deep learning is enabled:

```
final_score = (
    traditional_aggregated × 0.30 +
    deep_learning_aggregated × 0.25 +
    sift_score × 0.20 +
    ssim_aggregated × 0.15 +
    phash_best × 0.10
)
```

When deep learning is disabled (weight redistributed proportionally):

```
total_w = 0.30 + 0.20 + 0.15 + 0.10 = 0.75
final_score = (
    traditional_aggregated × (0.30/0.75) +
    sift_score × (0.20/0.75) +
    ssim_aggregated × (0.15/0.75) +
    phash_best × (0.10/0.75)
)
```

OCR Bonus

```
if ocr_match:
    final_score = min(100.0, final_score + 10.0)
```

Safety Check — Minimum Good Pairs

```
good_pair_count = sum(1 for s in traditional_scores if s >= 60.0)
if good_pair_count < 2 and final_score >= 85.0:
    final_score = min(final_score, 84.9)  # demote to PENDING
```

Rationale: If only one image pair out of nine is good but the final score somehow reaches 85%, the result is suspicious. The safety check requires at least 2 pairs to be above the manual-review threshold (60%) before allowing auto-approval.

---

6. Decision Flow

```
Input: original_images[], kiosk_images[], attempt_number
│
├── Stage 1: Quality Gate
│    ├── ALL kiosk images fail? ──────────────────► RETRY (retake photos)
│    └── Some/all pass ──► continue
│
├── Stage 2: pHash Pre-filter
│    ├── ALL pairs < 40% similarity? ─────────────► RETRY (attempt < 10)
│    │                                              REJECTED (attempt = 10)
│    └── Any pair ≥ 40%? ──► continue
│
├── Stage 3: Traditional CV
│    └── Compute all pairwise scores (9 pairs)
│
├── Stage 4: SIFT + RANSAC
│    └── Compute best inlier ratio + match ratio
│
├── Stage 5: SSIM
│    └── Compute all pairwise scores
│
├── Stage 6: ResNet50
│    └── Compute all pairwise cosine similarities
│
├── Stage 7: OCR
│    └── Match serial numbers (if found)
│
├── Stage 8: Hybrid Score
│    ├── Aggregate each method's scores (trimmed mean)
│    ├── Compute weighted sum
│    ├── Add OCR bonus (+10 if serial match)
│    ├── Safety check (min 2 good pairs)
│    │
│    ├── score ≥ 85.0 ────────────────────────► APPROVED
│    │                                            (payment released)
│    │
│    ├── 60.0 ≤ score < 85.0 ──────────────────► PENDING
│    │                                            (admin manual review)
│    │
│    └── score < 60.0
│         ├── attempt < 10 ────────────────────► RETRY
│         │                                      (reposition item)
│         └── attempt = 10 ────────────────────► REJECTED
│                                                (max attempts reached)
```

---

7. Libraries — Complete Reference

All libraries listed below are **general-purpose, industry-standard tools**. None of them are created for or exclusive to this project. They are used in applications ranging from medical imaging to self-driving cars to document scanning. What is unique to this project is *how they are combined* — see Section 8.

Web Framework

Library	Version	Role in this project	General-purpose?
FastAPI	0.115.6	HTTP server for the ML microservice. Handles request routing, file uploads, response serialisation.	Yes — widely used for Python APIs, especially ML services
uvicorn	0.34.0	ASGI server that runs FastAPI. Production-grade async HTTP server.	Yes — the standard server for FastAPI/Starlette apps
python-multipart	0.0.20	Enables parsing of `multipart/form-data` requests (required for file uploads in FastAPI).	Yes — required by FastAPI for any `File(...)` parameter
pydantic-settings	2.7.0	Type-safe configuration management from environment variables. All thresholds and settings in `config.py` are loaded through this.	Yes — used across Python microservices for config management

---

Computer Vision

Library	Version	Role in this project	General-purpose?
opencv-python-headless	4.10.0.84	The core CV library. Used for: SIFT keypoint detection, FLANN matching, RANSAC homography, GrabCut background removal, HOG descriptor, ORB keypoints, Canny edge detection, Hu moments, HSV/LAB color conversions, CLAHE lighting normalisation, image resizing, Laplacian blur detection, Gaussian blur, BFMatcher.	Yes — the most widely used open-source CV library. Used across industry and academia. `-headless` means no GUI dependencies (suitable for server deployment).
scikit-image	0.24.0	Used specifically for **Local Binary Pattern (LBP)** texture feature extraction (`local_binary_pattern()`). OpenCV does not have a built-in LBP implementation.	Yes — a Python image processing library complementary to OpenCV. Used in scientific imaging, medical imaging, remote sensing.
Pillow	11.0.0	Image loading and format conversion required by PyTorch's `transforms`. PyTorch's image transforms operate on PIL Images, so OpenCV's NumPy arrays must be converted via Pillow.	Yes — the standard Python imaging library. Used everywhere images need to be loaded/saved in Python.

Numerical Computing

Library	Version	Role in this project	General-purpose?
NumPy	1.26.4	Array operations throughout the entire pipeline: feature vector arithmetic, histogram normalisation, DCT for perceptual hashing, aggregation statistics, mask operations. Every feature vector is a NumPy array.	Yes — the foundational numerical computing library for Python. Used in virtually every scientific computing project.

SciPy	1.14.1	Two specific functions: `scipy.spatial.distance.cosine` (cosine distance for color, spatial, HOG, deep feature comparison) and `scipy.stats.pearsonr` (Pearson correlation for texture comparison).	Yes — the scientific computing library built on NumPy. Used across signal processing, statistics, optimisation.

Deep Learning

Library	Version	Role in this project	General-purpose?
PyTorch	2.5.1	The deep learning framework. Runs the ResNet50 model for inference. Provides tensors, `torch.no_grad()` context for inference mode, and the model execution engine.	Yes — one of the two dominant deep learning frameworks (alongside TensorFlow/Keras). Used extensively in both research and production.
TorchVision	0.20.1	Provides the pre-trained ResNet50 model (`models.resnet50(weights=ResNet50_Weights.IMAGENET1K_V2)`) and image transforms (`transforms.Resize`, `transforms.CenterCrop`, `transforms.ToTensor`, `transforms.Normalize`).	Yes — the PyTorch library for computer vision models. Contains pre-trained models (ResNet, VGG, EfficientNet, etc.) and dataset utilities.

Note on the ResNet50 model itself: The model weights (IMAGENET1K_V2) are downloaded automatically by TorchVision from PyTorch's model hub on first use. The weights are 100MB. The model was trained by Meta AI Research. It is **not** a custom-trained model — it is a standard pre-trained backbone re-purposed for feature extraction by removing its classification head.

Machine Learning Utilities

Library	Version	Role in this project	General-purpose?
scikit-learn	1.6.0	Available for pairwise metrics but the project primarily uses SciPy for similarity functions. Present as a utility dependency.	Yes — the standard Python ML library for classical algorithms.

OCR

Library	Version	Role in this project	General-purpose?
pytesseract	0.3.13	Python wrapper for Google's Tesseract OCR engine. Extracts text from item images to find serial numbers and model numbers.	Yes — the most widely used open-source OCR solution. Used in document scanning, receipt

		Requires the `tesseract-ocr` system package (installed in Dockerfile).	processing, form recognition worldwide.

Note on Tesseract: Tesseract itself is a C++ program maintained by Google, originally developed by HP Labs. `pytesseract` is just a thin Python wrapper that calls the `tesseract` command-line tool and parses its output. Both must be installed: `pip install pytesseract` + `apt-get install tesseract-ocr`.

Logging & Testing

Library	Version	Role in this project	General-purpose?
structlog	24.4.0	Structured JSON logging throughout the service. Each log entry is a JSON object with fields like `event`, `stage`, `confidence`, etc. — easier to query in log aggregation tools.	Yes — used in production Python services that need queryable logs.
pytest	8.3.4	Test framework (test suite not yet written, present in requirements).	Yes — the standard Python testing framework.
pytest-asyncio	0.24.0	Enables testing of async FastAPI routes.	Yes — extension for testing async Python code.
httpx	0.28.1	HTTP client used by FastAPI's test client for integration testing.	Yes — an async-capable HTTP client for Python.

---

8. What Is Unique to This Project

The libraries described above are all used in many other projects worldwide. **The uniqueness of this system lies in the combination and application**, not the individual tools.

1. The 8-Stage Hybrid Pipeline Architecture

No existing off-the-shelf solution combines all of these methods in a sequential pipeline with stage-specific early exits. The design choices are deliberate:

- **Quality gate first** — no point running expensive algorithms on a blurry image
- **pHash as fast pre-filter** — O(1) operation that eliminates obvious non-matches before any expensive computation
- **Traditional CV before deep learning** — fast and interpretable, catches obvious cases cheaply
- **RANSAC validation** — geometric verification that prevents false positives from happening
- **Deep learning last** — most computationally expensive, only runs when other methods are inconclusive

2. Two-Checkpoint Bidirectional Fraud Prevention

The design of calling the same verification service at both deposit AND return is specific to this rental use case. This provides symmetric protection:

- Owner cannot swap items at deposit (renter protection)
- Renter cannot swap items at return (owner protection)

3. Trimmed-Mean Score Aggregation

The decision to use trimmed mean (drop bottom 20%, top 10%) instead of max or average for aggregating multiple image pair scores is a deliberate design choice to prevent gaming:

- Cannot game with one great photo among bad ones (trimmed mean discards outliers)
- Cannot be penalised by one bad photo among good ones (bottom trimmed)

4. Minimum Good Pairs Safety Check

Requiring at least 2 image pairs to exceed 60% before allowing auto-approval is a second layer of fraud prevention. It prevents the case where one coincidentally good pair inflates the score above 85%.

5. Kiosk-Specific Background Removal

The white-threshold background removal method is designed specifically for the locker environment. The locker interior is white (V > 200 in HSV), making a simple threshold effective where GrabCut might struggle with a uniformly white background.

6. Configurable Weights for All Methods

Every weight and threshold is configurable via environment variables. This allows the system to be tuned as real-world usage data is collected, without code changes.

7. Feature Caching via Item.mlFeatures

Pre-extracting traditional and deep features when an item is listed, storing them in the database, and reusing them for every subsequent verification avoids running ResNet50 twice per verification. This is a performance optimisation specific to this database-backed workflow.

8. The Application Domain

Campus equipment rental fraud prevention using kiosk-captured images is a novel application. The specific item categories (lab gowns, calculators, Arduino kits, power banks) and the environmental challenge (home photo vs. locker photo) are unique to this system.

---

9. Configuration Reference

All parameters are configurable via environment variables (`.env.example`). Defaults are production-ready values.

Service Settings

Variable	Default	Description
`DEBUG`	`false`	Enable debug logging
`HOST`	`0.0.0.0`	Listen address
`PORT`	`8001`	Listen port

Verification Thresholds

Variable	Default	Description
`THRESHOLD_VERIFIED`	`85.0`	Score ≥ this → APPROVED (auto)
`THRESHOLD_MANUAL_REVIEW`	`60.0`	Score ≥ this → PENDING (admin review)

MAX_RETRY_ATTEMPTS	10	Max RETRY decisions before REJECTED

Traditional CV Feature Weights (must sum to 1.0)

Variable	Default	Feature
WEIGHT_COLOR	0.22	HSV color histogram
WEIGHT_SPATIAL	0.13	Spatial color pyramid
WEIGHT_SHAPE	0.12	Hu moments
WEIGHT_TEXTURE	0.13	LBP texture
WEIGHT_HOG	0.18	HOG edge gradients
WEIGHT_ORB	0.12	ORB keypoint matching
WEIGHT_SSIM	0.10	SSIM (used within traditional stage)

Hybrid Method Weights (must sum to 1.0)

Variable	Default	Method
WEIGHT_TRADITIONAL	0.30	Traditional CV aggregated score
WEIGHT_DEEP_LEARNING	0.25	ResNet50 cosine similarity
WEIGHT_SIFT	0.20	SIFT + RANSAC combined score
WEIGHT_SSIM_HYBRID	0.15	SSIM structural similarity
WEIGHT_PHASH_HYBRID	0.10	Best perceptual hash score

Feature Extraction Settings

Variable	Default	Description
ORB_FEATURES_COUNT	200	Max ORB keypoints per image
SIFT_RATIO_THRESHOLD	0.7	Lowe's ratio test threshold
LBP_POINTS	8	LBP neighbour points (base scale)
LBP_RADIUS	1	LBP radius (base scale)
COLOR_HIST_BINS	32	S and V channel histogram bins

Quality Gate Thresholds

Variable	Default	Description
QUALITY_MIN_BLUR_SCORE	50.0	Laplacian variance minimum
QUALITY_MIN_BRIGHTNESS	40.0	Mean pixel brightness minimum

`QUALITY_MAX_BRIGHTNESS`	`240.0`	Mean pixel brightness maximum
`QUALITY_MIN_COVERAGE`	`0.05`	Item foreground coverage minimum (5%)

Pre-filter & Aggregation

Variable	Default	Description
`PHASH_OBVIOUS_MISMATCH_THRESHOLD`	`40.0`	pHash below this = obvious mismatch
`MIN_GOOD_PAIRS`	`2`	Minimum pairs ≥ 60% to allow auto-approval
`SCORE_AGGREGATION`	`trimmed_mean`	`max`, `median`, or `trimmed_mean`

Feature Toggles

Variable	Default	Description
`ENABLE_DEEP_LEARNING`	`true`	Enable ResNet50 (disable to reduce memory/compute)
`ENABLE_OCR`	`true`	Enable pytesseract OCR serial matching

Image Processing

Variable	Default	Description
`MAX_IMAGE_SIZE`	`4096`	Images larger than this are not loaded
`UPLOAD_DIR`	`/tmp/engirent_uploads`	Temp directory for uploaded files

---

10. API Endpoints

Base URL: `http://localhost:8001/api/v1`

POST /verify

Full 8-stage hybrid verification.

Content-Type: `multipart/form-data`

Parameters:

Field	Type	Required	Description
`original_images`	file[]	Yes	Owner's listing photos (minimum 1, recommended 3+)
`kiosk_images`	file[]	Yes	Kiosk camera captures (minimum 1, recommended 3–5)
`attempt_number`	int	No (default: 1)	Current attempt number (1–10). Backend passes `depositAttemptCount + 1` or `returnAttemptCount + 1`.

`reference_features`	string	No	JSON-encoded pre-extracted features from `Item.mlFeatures`. If provided, skips traditional and deep feature re-extraction for original images.

Response: See Section 11

POST /extract-features

Pre-extract features from listing images for caching in `Item.mlFeatures`.

Content-Type: `multipart/form-data`

Parameters:

Field	Type	Required	Description
`images`	file[]	Yes	Item listing photos

Response:

```
{
  "image_count": 3,
  "traditional_features_count": 3,
  "deep_features_count": 3,
  "ocr_texts": ["S/N ABC123", "", "CASIO FX-991EX"],
  "features": {
    "traditional": [...],
    "deep": [...],
    "ocr_texts": ["S/N ABC123", "", "CASIO FX-991EX"],
    "image_count": 3
  }
}
```

The `features` object is stored directly in `Item.mlFeatures` in the database.

GET /health

Service health check.

Response:

```
{
  "status": "healthy",
  "service": "EngiRent AI Verification Service",
  "deep_learning_enabled": true,
  "ocr_enabled": true
}
```

---

11. API Response Structure

Full response from `POST /verify` :

```
{
  "verified": true,
  "decision": "APPROVED",
  "message": "Item verified successfully.",
  "confidence": 87.34,
  "attempt_number": 1,

  "method_scores": {
    "traditional_best": 91.20,
    "traditional_aggregated": 88.50,
    "sift_best_match": 72.40,
    "sift_best_inlier": 68.90,
    "sift_combined": 71.09,
    "ssim_aggregated": 83.20,
    "deep_learning_aggregated": 92.10,
    "phash_best": 85.50
  },

  "ocr": {
    "match": true,
    "details": {
      "original_serials": ["SN-ABC123", "CASIO FX-991EX"],
      "kiosk_serials": ["SN-ABC123"],
      "matched_serials": ["SN-ABC123"]
    }
  },

  "quality_issues": [
    {
      "image_index": 2,
      "passed": false,
      "blur_score": 38.2,
      "brightness": 180.0,
      "coverage_percent": 0.45,
      "issues": ["Image is too blurry (score: 38.2, minimum: 50.0)"]
    }
  ],

  "good_pair_count": 6,
  "all_traditional_scores": [88.2, 91.2, 85.4, 90.1, 87.3, 88.9, 82.1, 88.0, 91.0],
  "sift_all_ratios": [68.2, 72.4, 65.1, 70.8, 71.2, 69.0, 63.4, 68.9, 72.0]
}
```

Decision values:

Value	Meaning	Backend action
APPROVED	Confidence ≥ 85%	Release payment / advance rental

`PENDING`	60% ≤ confidence < 85%	Create manual review task for admin
`RETRY`	Confidence < 60%, attempts remaining	Release locker, prompt repositioning
`REJECTED`	Confidence < 60%, max attempts reached	Cancel rental (deposit) / dispute (return)

---

12. Feature Caching

The Problem

ResNet50 inference takes ~500ms per image. If an item has 3 listing photos, re-extracting deep features at every verification costs ~1.5 seconds just for the original images — every time. With traditional CV, the total re-extraction time for 3 original images is ~2–3 seconds per verification call.

The Solution

When an item is created (`POST /api/v1/items`), the backend (`itemController.ts`) triggers a **non-blocking background call** to `POST /api/v1/extract-features` using `setImmediate()` . The response — containing traditional CV features, ResNet50 feature vectors, and OCR texts — is stored in `Item.mlFeatures` (a `Json?` column in the database).

```
createItem() HTTP request → response sent to user immediately
     │
     └──(setImmediate)──► download item images
                          POST /api/v1/extract-features
                          prisma.item.update({ mlFeatures: response.features })
                          log: "ML features cached for item {id}"
```

How It's Used

In both `depositItem()` and `returnItem()` , before building the FormData:

```
if (rental.item.mlFeatures) {
  formData.append('reference_features', JSON.stringify(rental.item.mlFeatures));
}
```

The ML service router parses the JSON and passes it to `verifier.verify(..., reference_features=parsed_features)` . Inside `hybrid.py` :

```
if reference_features and "traditional" in reference_features:
    orig_traditional = reference_features["traditional"]  # skip re-extraction
else:
    orig_traditional = self.traditional.extract_batch(original_sources)  # extract fresh

if reference_features and "deep" in reference_features and reference_features["deep"]:
    orig_deep = [np.array(f) for f in reference_features["deep"]]  # skip ResNet50
else:
    orig_deep = self.deep.extract_batch(original_sources)  # run ResNet50
```

Time saved: ~2–3 seconds per verification call when features are cached.

Failure handling: If the background extraction fails (ML service down when item is created), `mlFeatures` stays `null` and every verification re-extracts from scratch. This is a graceful degradation — slower but still correct.

---

13. Backend Integration

How Node.js Calls the Service

The Node.js backend (`server/src/controllers/kioskController.ts`) communicates with the ML service via HTTP using `axios` .

Image delivery: Images are stored as URLs in the database. Before calling the ML service, the backend downloads each image URL to a `Blob` :

```
const response = await axios.get(url, { responseType: 'arraybuffer' });
const blob = new Blob([response.data], { type: 'image/jpeg' });
formData.append('original_images', blob, 'original_0.jpg');
```

This converts stored URLs to actual file bytes that FastAPI's `File(...)` parameters can receive.

Environment Variables (Backend)

```
ML_SERVICE_URL=http://localhost:8001
ML_SERVICE_API_KEY=optional-api-key     # sent as X-Api-Key header if set
```

Database Models Involved

`Verification` table — one record created per verification event:

```
model Verification {
  id                String               // UUID
  originalImages    Json                 // Array of listing image URLs
  kioskImages       Json                 // Array of kiosk image URLs
  decision          VerificationDecision // APPROVED / PENDING / RETRY / REJECTED
  confidenceScore   Float                // Final hybrid score (0-100)
  attemptNumber     Int                  // Which attempt this was
  traditionalScore  Float?               // method_scores.traditional_best
  siftScore         Float?               // method_scores.sift_combined
  deepLearningScore Float?               // method_scores.deep_learning_aggregated
  ocrMatch          Boolean?             // ocr.match
  ocrDetails        Json?                // ocr.details
  status            VerificationStatus   // APPROVED / MANUAL_REVIEW / REJECTED
  reviewedBy        String?              // Admin ID (if manually reviewed)
  reviewNotes       String?              // "ML service unavailable: ..." or admin notes
}
```

`Rental` table — links to two Verification records and tracks attempt counts:

```
depositVerificationId  String?   // FK to Verification (deposit checkpoint)
verificationId         String?   // FK to Verification (return checkpoint)
```

```
depositAttemptCount    Int       // Incremented on each deposit RETRY/REJECTED
returnAttemptCount     Int       // Incremented on each return RETRY/REJECTED
verificationScore      Float?    // Final score of most recent verification
verificationStatus     VerificationStatus?
```

`Item` **table** — caches pre-extracted features:

```
mlFeatures  Json?   // { traditional: [...], deep: [...], ocr_texts: [...], image_count: N }
```

---

14. Deployment

Dockerfile

```
FROM python:3.12-slim

# System dependencies
RUN apt-get update && apt-get install -y \
    libgl1-mesa-glx \    # OpenCV GUI dependencies (even headless needs libGL)
    libglib2.0-0 \       # GLib (OpenCV dependency)
    tesseract-ocr \      # Google Tesseract OCR engine (required by pytesseract)
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app/ app/
EXPOSE 8001
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8001"]
```

Key system packages:

- `libgl1-mesa-glx` — OpenCV requires `libGL.so.1` even in headless mode
- `libglib2.0-0` — OpenCV dependency for threading and GLib utilities
- `tesseract-ocr` — the Tesseract OCR binary that pytesseract wraps

Resource Requirements

Resource	Minimum	Recommended
RAM	2 GB	4 GB (ResNet50 model = ~100MB weights + ~500MB runtime)
CPU	2 cores	4 cores
Disk	2 GB	4 GB (PyTorch + TorchVision + OpenCV + model weights)
GPU	Not required	Optional (PyTorch uses CPU inference by default)

Startup Time

- Cold start: ~5–8 seconds (loading Python, FastAPI, importing OpenCV)

- First verification: +2–3 seconds for lazy ResNet50 model load
- Subsequent verifications: ResNet50 already loaded, runs from memory

Ports

- ML Service: `8001`
- Node.js API: `5000`
- Admin Console: `3001`
- Web App: `3000`

---

15. File Structure

```
server/python_server/services/ml/
│
├── requirements.txt          # All Python dependencies (pinned versions)
├── Dockerfile                # Container definition
├── .env.example              # All configurable environment variables
│
└── app/
    ├── __init__.py
    ├── main.py               # FastAPI app setup, CORS, router registration
    ├── config.py             # Settings class (pydantic-settings), all defaults
    │
    ├── models/
    │   ├── __init__.py
    │   └── schemas.py        # Pydantic request/response models
    │                         #   VerificationResponse, MethodScores, OCRResult
    │                         #   QualityIssue, FeatureExtractionResponse
    │                         #   StorableFeatures, HealthResponse
    │
    ├── routers/
    │   ├── __init__.py
    │   └── verification.py   # FastAPI route handlers
    │                         #   POST /verify
    │                         #   POST /extract-features
    │                         #   GET  /health
    │
    ├── comparison/
    │   ├── __init__.py
    │   ├── hybrid.py         # Main pipeline engine (HybridVerifier)
    │   │                     #   verify() — 8-stage pipeline
    │   │                     #   extract_reference_features()
    │   │                     #   _aggregate_scores() — trimmed mean
    │   │                     #   _make_decision() — threshold logic
    │   └── similarity.py     # Feature comparison math (SimilarityCalculator)
    │                         #   compare_traditional() — 6-method weighted score
    │                         #   compare_deep() — cosine similarity
    │                         #   compare_ssim() — Wang et al. 2004
    │                         #   _orb_descriptor_match() — BFMatcher + ratio test
    │                         #   _cosine_similarity(), _shape_similarity()
    │                         #   _correlation_similarity()
```

```
    │
    ├── features/
    │   ├── __init__.py
    │   ├── traditional.py    # TraditionalFeatureExtractor
    │   │                     #   extract() — single image
    │   │                     #   extract_batch() — multiple images
    │   │                     #   _color_histogram_hsv() — 84-d
    │   │                     #   _spatial_color_pyramid() — 108-d
    │   │                     #   _shape_descriptors() — 7-d Hu moments
    │   │                     #   _texture_lbp_multiscale() — 62-d
    │   │                     #   _hog_features() — variable-d
    │   │                     #   _orb_raw_descriptors() — N×32
    │   ├── sift.py           # SIFTFeatureExtractor
    │   │                     #   detect_keypoints()
    │   │                     #   match() — FLANN + ratio test + RANSAC
    │   │                     #   match_multi() — all pairwise combinations
    │   ├── deep.py           # DeepFeatureExtractor (ResNet50)
    │   │                     #   extract() — single image → 2048-d vector
    │   │                     #   extract_batch() — multiple images
    │   │                     #   _load_model() — lazy loader
    │   └── phash.py          # Perceptual hash functions
    │                         #   compute_phash() — DCT hash
    │                         #   compute_dhash() — gradient hash
    │                         #   hamming_distance()
    │                         #   phash_similarity() — 0-100 score
    │                         #   is_obvious_mismatch() — fast boolean check
    │
    └── utils/
        ├── __init__.py
        ├── image.py          # Image I/O and normalisation
        │                     #   load_image() — path/bytes/array
        │                     #   white_balance() — gray-world
        │                     #   normalize_lighting() — CLAHE
        │                     #   resize_image() — aspect-ratio preserving
        │                     #   preprocess() — full pipeline
        ├── background.py     # Background removal
        │                     #   remove_background_grabcut() — GrabCut
        │                     #   remove_background_kiosk() — white threshold
        │                     #   _subtract_background() — diff from empty frame
        │                     #   _threshold_white_background()
        │                     #   get_item_crop() — tight bounding box
        ├── quality.py        # Image quality gate
        │                     #   check_quality() — blur + brightness + coverage
        │                     #   QualityCheckResult — dataclass
        └── ocr.py            # Text extraction and serial matching
                              #   extract_text() — pytesseract wrapper
                              #   find_serial_numbers() — regex patterns
                              #   match_serial_numbers() — set intersection
```